JS8Mesh User Manual

1. What JS8Mesh Is

JS8Mesh is a human-supervised mesh-awareness and relay tool for JS8Call.

It is **not** an automated routing mesh network. Every transmission still depends on a human operator deciding what to send and when to send it.

JS8Mesh helps you:
- see who hears whom
- compare likely relay paths
- prepare relay text for JS8Call
- generate mesh reports
- answer supported `JC` requests with prepared replies

2. Important Disclaimer

Use according to your local laws. Use at your own risk.

The operator is responsible for all transmissions.

JS8Mesh was built with the help of AI and should be treated as experimental software.

Always supervise your station.

3. What You Need

- Windows or Linux
- JS8Call installed and working
- a valid callsign configured in JS8Mesh
- activity/traffic in JS8Call so JS8Mesh has data to work with

Optional but recommended:
- JS8Call Control enabled in JS8Mesh if you want JS8Mesh to send/stage text to JS8Call automatically

If you are running JS8Mesh from source instead of a packaged release, you also need:
- Python
- `pyjs8call`
- Tk/Tkinter support
- run the app from `main.py`

Compatibility note:
- JS8Mesh is designed for JS8Call versions that provide `DIRECTED.TXT`-compatible directed-message output
- compatibility should not be assumed with every historical JS8Call version or

every fork/build unless that behavior is present and compatible

Typical Windows `DIRECTED.TXT` location:
- `%LOCALAPPDATA%\JS8Call\DIRECTED.TXT`
- usually resolves to `C:\Users\<YourUser>\AppData\Local\JS8Call\DIRECTED.TXT`

4. First Start

Important setup notes:
- start JS8Call first, then start JS8Mesh
- add `@JS8MESH` to your JS8Call groups

When JS8Mesh opens for the first time:

1. Open `Settings`.
2. Set your callsign information.
3. Find `DIRECTED.TXT`.
4. Choose or add the operating frequency you want.
5. Set the maximum TX time everywhere it appears in `Settings`.
6. Decide whether `JS8Call Control` should be `YES` or `NO`.

If `JS8Call Control` is `YES`, JS8Mesh can:
- load prepared text into JS8Call
- send to JS8Call through its integration path
- try to change JS8Call speed mode before send
- try to restore the previous mode after transmission

If `JS8Call Control` is `NO`, JS8Mesh only prepares/stages text and you remain fully manual in JS8Call.

5. Main Window

The main window is centered around a few main areas:

- top controls
  - user callsign
  - target callsign
  - operating frequency
  - saved settings
- `Linear Pathways`
  - recommended relay routes to the current target
- `Inbound Reachability`
  - stations that may help inbound convergence
- `RX Monitor`
  - mirror of the JS8Call receive monitor
- `Relay Profiles`
  - summary view of known relay stations
- `Topology`
  - traffic or mesh structure view

- `Activity`
  - decoded records seen by JS8Mesh
- `Relay Message Builder`
  - prepare relay messages for JS8Call

6. Main Concepts

Nodes and Stations

- `node`
  - a station using JS8Mesh and participating in structured mesh reports
- `station`
  - any callsign seen/heard in records, even if it is not using JS8Mesh

Linear Pathway

A `Linear Pathway` is a relay route JS8Mesh considers usable based on its stored hearing/report evidence.

In a linear pathway, the stored evidence supports the idea that each station in the chain can hear the station before it and the station after it in a way that fits JS8Mesh pathway logic.

That is why a linear pathway is the safer and more trusted route type. It is based on evidence that more directly supports the whole relay chain.

Inbound Convergence

`Inbound Convergence` is different from a linear pathway.

It represents a station or route that may help messages converge toward the target, but the route is not as directly proven as a native linear pathway.

So:
- `Linear Pathway`
  - stronger route evidence
  - safer for actual relay use
  - preferred when available
- `Inbound Convergence`
  - useful hint or candidate
  - may later become stronger with more evidence or manual success
  - less directly proven than a linear pathway

In simple terms:
- `Linear` means the chain is supported more directly by what JS8Mesh knows
- `Inbound` means the route may help, but it is not as fully proven

Direct

`Direct` means JS8Mesh sees a direct path between you and the target according to its evidence model.

Freshness and SNR

JS8Mesh prefers:
- fresher evidence
- stronger SNR
- shorter relay pathways

Freshness is divided into 4 stages:
- `0-10 min`
- `10-30 min`
- `30-65 min`
- `>65 min`

For normal pathway generation, evidence older than 65 minutes is treated as too old to use. In practical terms, after that point the station/path is treated as QRT for pathway generation unless you are using Test Mode or newer evidence appears.

7. Settings

Callsign

Use `Settings > Callsign` to:
- set your amateur radio callsign
- optionally set a special callsign
- assign that special callsign to a specific frequency if you need to

Add / Remove Frequency

Use `Settings > Add / Remove Frequency` to:
- add custom frequencies
- remove custom frequencies you no longer want

JS8Call Control

Use `Settings > JS8Call Control` to choose whether JS8Mesh should only prepare text or also hand it off to JS8Call.

Sync Frequency from JS8Call

Use `Settings > Sync Frequency from JS8Call` if you want JS8Mesh to follow the frequency currently used in JS8Call.

JR/HR TX Time Limit

Use `Settings > JR/HR TX Time Limit` to set the default transmit-time limit for

generated:

- `JR`
- `JRN`
- `JRS`
- `HR`
- `HRC`

If a generated reply would exceed the limit, JS8Mesh trims or refuses it as needed.

Important note:
The result of the JS8Mesh TX time estimator may differ from the actual TX time JS8Call will need. Always be cautious. Better safe than sorry. Use at your own risk.

Watch Callsigns

Use `Settings > Watch Callsigns` to add callsigns that should trigger an on-screen notification when they appear in new activity.

Test Mode

`Test Mode` is useful when you want JS8Mesh to work from a limited slice of recent activity instead of normal freshness rules.

The `recent records` box means:
- the number of most recent lines from `DIRECTED.TXT` that JS8Mesh will use while Test Mode is ON

So if `recent records` is set to `100`, Test Mode works from the latest 100 decoded records instead of the normal live freshness window.

If `recent records` is set to `0`, JS8Mesh will use all of `DIRECTED.TXT`. Practically, that means all information ever noted there.

8. Mesh Reports Menu

Request JR/HR

Use `Mesh Reports > Request JR/HR` to send these request types:

- `JR`
  - general structured mesh report
- `JRN`
  - nodes only
- `JRS`
  - stations only
- `HR`

  - direct-heard station snapshot
- `HRC`
  - ask whether the node can directly communicate with a specific callsign

Requests can be:
- direct to a node
- relayed to a node

TX Mesh Reports

Use `Settings > TX Mesh Reports` to set:
- station count
- lookback period
- interval or fixed schedule
- mode
- TX time limit

When a scheduled mesh report is due, JS8Mesh prepares the report and shows the confirmation/send flow according to current settings.

9. Supported Request Types

JR

Returns a structured multi-wave mesh report.

It will include at least one node if available.

JRN

Same structured report style as `JR`, but nodes only.

JRS

Same structured report style as `JR`, but stations only.

HR

Returns up to 4 directly heard stations using JS8Mesh selection criteria.

HRC

Asks whether the node can directly communicate with one specific callsign.

If yes, JS8Mesh prepares a one-line `JR` reply about that callsign only.

If no direct communication is found, no reply is generated.

Relayed Requests

Direct and relayed requests are supported for:
- `JR`
- `JRN`
- `JRS`
- `HR`
- `HRC`

If a request arrives by relay and a reply is generated, the reply follows the reverse path.

10. How To Read Reports

Here are a few simple examples using fake callsigns.

Example 1: JR

`JR.1.calls1.+10.3;2.calls2.calls1.+8.9;3.calls3.calls2.+6.15`

Read it like this:
- `1.calls1.+10.3`
  - wave 1
  - `calls1` is directly reported first
  - `+10` is the SNR
  - `3` means about 3 minutes old
- `2.calls2.calls1.+8.9`
  - wave 2
  - `calls2` is behind `calls1`
  - `+8` is the SNR
  - `9` means about 9 minutes old
- `3.calls3.calls2.+6.15`
  - wave 3
  - `calls3` is behind `calls2`
  - `+6` is the SNR
  - `15` means about 15 minutes old

So the structure is:
- responder -> `calls1` -> `calls2` -> `calls3`

Example 2: HR

`HR.1.*calls1.+12.2;1.calls2.+9.4;1.calls3.+7.8`

Read it like this:
- every entry is `wave 1`
- these are stations heard directly by the responding node
- `*calls1` means `calls1` is recognized as a node
- `+12`, `+9`, `+7` are SNR values
- `2`, `4`, `8` are freshness in minutes

So this means:
- the responding node directly hears `calls1`, `calls2`, and `calls3`
- `calls1` is a node

Example 3: HRC

`JR.1.calls4.+11.2`

An `HRC` reply is a very small targeted reply.

Read it like this:
- the requested node can directly communicate with `calls4`
- the evidence is wave 1 only
- `+11` is the SNR
- `2` means about 2 minutes old

If no direct communication is found for the searched callsign, no `HRC` reply is generated.

11. Requested JR Window

When JS8Mesh can prepare a reply for an incoming request, it opens the `Requested JR` window directly.

This window shows:
- who requested the report
- the request path
- the requested target callsign for `HRC`, when applicable
- prepared reply text
- estimated TX time
- default speed mode
- what JS8Mesh will try to do in JS8Call before send

Buttons:
- `Send to JS8Call`
- `Copy to Clipboard`
- `Close`

If no report can be generated, JS8Mesh skips quietly and records the result in the appropriate log where applicable.

12. Relay Message Builder

The `Relay Message Builder` lets you:
- select a pathway from `Linear Pathways`
- type a message
- see the fully prepared relay text
- choose TX mode

- send/stage the result to JS8Call

The `Default = ...` text on the `TX MODE` line shows the default speed mode for the currently selected pathway.

Mark Success, Mark Failure, Pending, and ACK

When you send a relay message, JS8Mesh can track the attempt in `Past Relays Log`.

Possible states:
- `P`
  - pending
- `S`
  - success
- `F`
  - failure

`Pending` means JS8Mesh is waiting to see whether an ACK-like reply appears.

ACK-like positive replies include tokens such as:
- `RR`
- `QSL`
- `ACK`
- `YES`
- `FB`
- `RGR`
- `ROGER`

If the expected ACK arrives in the correct return direction, JS8Mesh can mark the relay automatically as success.

If no matching ACK arrives in time, the relay is automatically marked as failure unless the operator changes it manually first.

Timeout rule:
- JS8Mesh waits until the estimated transmission time has passed
- then it adds a 5-minute ACK recognition window
- after that, the pending relay is marked as `F` unless:
  - a matching ACK was seen, or
  - the operator manually marked it as success or failure

The operator can also use:
- `Mark Success`
- `Mark Failure`

to override the pending result manually.

Important:

Marking `Success` is very useful because it affects future pathway ranking. Pathways that have proven successful can move higher in the list, helping JS8Mesh prefer pathways that worked in real use instead of only looking at hearing evidence.

13. Topology

The `Topology` window has two views:

- `traffic`
  - overall recent traffic observations
- `mesh`
  - decoded mesh structure from reports such as `JR` and `HR`

Mesh wave filters can show:
- all waves
- exact wave filters
- deeper filters such as `ONLY 9+`

Incoming `HR` reports are mirrored into mesh topology as direct-hearing edges.

14. Loggers

Use `Loggers` to open:

- `Requested JR Responds Log`
- `TX Mesh Reports Log`
- `HR Log`
- `Past Relays Log`

These windows support:
- row selection
- right-click copy
- right-click select all
- export `.txt`
- export `.csv` where applicable
- clear log where applicable

15. RX Monitor

Use the `RX Monitor` button on the main window to view the JS8Call receive monitor inside JS8Mesh.

Behavior:
- opens at the latest line
- follows new lines at the bottom
- stops auto-follow if you scroll away manually

16. Maintenance

JS8Mesh keeps some logs/history and may occasionally prompt for maintenance.

When maintenance runs, entries older than 90 days are removed from retained log files.

17. GitHub / Release Use

If you are using the packaged `.exe` release:
- you do not need to install `pyjs8call` separately

If you are running the source code:
- you need Python
- you need `pyjs8call`
- you need Tk/Tkinter support
- the source entry point is `main.py`
- on Linux, you should expect to run JS8Mesh from source unless you create your own packaged build

Simple source-run examples:

Windows PowerShell:
```powershell
pip install pyjs8call
python main.py
```

Linux:
```bash
python3 -m pip install pyjs8call
python3 main.py
```

18. License

JS8Mesh is licensed under `GPL-3.0-only`.

Modified versions and forks should use a different name to avoid confusion with the original project.

See:
- LICENSE
- NAME_USE.md

19. Credits

- Uses `pyjs8call` for JS8Call integration.
- `"JC"` concepts were inspired in part by JS8Spotter.